CONDOSMART

CondoSmart

Plataforma Inteligente de Gestão de Condomínios

“Tudo o que seu condomínio precisa, em um só lugar.”

BLOCO 1 — CONCEPÇÃO E ESCOPO

1.1 Documento de visão

Nome do produto: CondoSmart — Plataforma Inteligente de Gestão de Condomínios.

Slogan: “Tudo o que seu condomínio precisa, em um só lugar.”

Problema: em muitos condomínios residenciais, informações sobre moradores, visitantes, reservas, comunicados, ocorrências e cobranças ficam distribuídas em planilhas, mensagens e registros manuais. Essa fragmentação dificulta o acompanhamento das solicitações, aumenta o retrabalho e reduz a transparência da administração.

Proposta de valor: centralizar as principais rotinas administrativas e de comunicação em uma plataforma web, com acesso controlado por perfil, histórico das operações e informações organizadas por unidade residencial.

Objetivo: oferecer aos moradores e à administração um canal digital único para consultar informações, solicitar serviços e acompanhar atividades do condomínio.

Escopo da primeira versão

- Cadastro e gerenciamento de usuários, moradores e unidades residenciais.
- Autorização prévia de visitantes e registro de entrada e saída pela portaria.
- Reserva e cancelamento de áreas comuns.
- Publicação de comunicados e envio de notificações.
- Registro e acompanhamento de ocorrências e solicitações de manutenção.
- Gerenciamento de funcionários e cobranças condominiais.
- Consulta de informações financeiras e geração de relatórios administrativos.

Fora do escopo

- Automação física de portões, elevadores ou catracas.
- Reconhecimento facial e monitoramento de câmeras.
- Integração bancária ou processamento direto de pagamentos na primeira versão.
- Aplicativo móvel nativo; o acesso inicial será por interface web responsiva.
- Gestão de condomínios comerciais ou mistos.

1.2 Personas

Persona	Idade e contexto	Objetivos e necessidades	Dificuldades atuais
Mariana Oliveira	29 anos; analista administrativa e moradora.	Reservar áreas, autorizar visitantes, ler comunicados e acompanhar solicitações.	Informações espalhadas em grupos de mensagens e pouca visibilidade do andamento dos pedidos.

Roberto Almeida	45 anos; administrador de empresas e síndico.	Administrar moradores, ocorrências, cobranças, comunicados e relatórios.	Controles manuais, retrabalho e dificuldade para consolidar dados.
Carlos Ferreira	38 anos; porteiro do condomínio.	Consultar autorizações e registrar entradas e saídas com rapidez.	Dependência de confirmações por telefone ou mensagens e registros manuais.

1.3 Mapa de Stakeholders

Stakeholder	Interesse principal	Influência
Moradores	Conveniência, segurança e acesso às informações.	Média
Síndico	Controle administrativo, financeiro e operacional.	Alta
Porteiros	Consulta de autorizações e registro de acessos.	Média
Funcionários	Recebimento e acompanhamento de atividades.	Média
Administrador da plataforma	Gestão de usuários, configurações e segurança.	Alta
Conselho fiscal	Transparência e consulta de informações financeiras.	Média
Equipe de desenvolvimento	Construção, manutenção e evolução do produto.	Alta

1.4 Estudo de viabilidade

Viabilidade técnica

A solução é tecnicamente viável com aplicação web React, backend Java 21 com Spring Boot, banco relacional PostgreSQL e API REST. A escolha preliminar atende às operações transacionais e ao controle de permissões. As decisões serão formalizadas nos ADRs do Bloco 3.

Viabilidade operacional

Moradores, síndicos, porteiros e funcionários poderão acessar o sistema por navegador em computadores, tablets ou smartphones. A implantação deverá incluir treinamento inicial, manual de uso, canal de suporte e migração controlada dos cadastros existentes.

Viabilidade financeira — infraestrutura

A estimativa cloud foi atualizada com os preços públicos do Amazon Lightsail consultados em 08/11/2026. Para produção, considera-se duas instâncias Linux de 4 GB (US$ 24 cada), balanceador de carga (US$ 18), banco PostgreSQL gerenciado de 4 GB em alta disponibilidade (US$ 120) e armazenamento de objetos de 100 GB (US$ 3). Total de referência: US$ 189/mês. A franquia de transferência da região São Paulo é reduzida em relação a outras regiões. Não estão incluídos impostos, domínio, excedentes, snapshots extras ou suporte.

Ambiente	Composição	Estimativa mensal
Desenvolvimento	1 instância Linux 2 GB para ambiente compartilhado; banco em container local.	US$ 12

Homologação	1 instância Linux 4 GB (US$ 24) + PostgreSQL gerenciado padrão 1 GB (US$ 15) + bucket 5 GB (US$ 1).	US$ 40
Produção	2 instâncias Linux 4 GB (US$ 48) + load balancer (US$ 18) + PostgreSQL gerenciado HA 4 GB (US$ 120) + bucket 100 GB (US$ 3).	US$ 189
Total estimado	Ambientes ativos simultaneamente; valores de lista em USD.	US$ 241/mês

A configuração de produção possui redundância das instâncias de aplicação e do banco gerenciado em alta disponibilidade. A meta de disponibilidade definida é 99,5% ao mês, mas ela não deve ser tratada como SLA garantido sem monitoramento, testes de failover e avaliação dos componentes externos.

1.5 Riscos iniciais

Risco	Probabilidade	Impacto	Mitigação
Vazamento de dados pessoais	Média	Alto	Criptografia, RBAC, auditoria e revisão de permissões.
Indisponibilidade da aplicação	Média	Alto	Balanceador, instâncias redundantes, alertas e plano de recuperação.
Perda de dados	Baixa	Alto	Backups automatizados, retenção e testes de restauração.
Acesso indevido por falha de autenticação	Média	Alto	Spring Security, JWT, limitação de tentativas e testes de segurança.
Baixa adesão dos moradores	Média	Médio	Treinamento, comunicação e interface responsiva.
Falha na autorização de visitante	Média	Alto	Validação de validade, unidade e responsável; trilha de auditoria.

BLOCO 2 — REQUISITOS

2.1 Requisitos funcionais (RF)

ID	Requisito	Prioridade e justificativa
RF001	Cadastrar moradores.	Must — base do cadastro residencial.
RF002	Cadastrar e editar unidades residenciais.	Must — organiza os vínculos do condomínio.
RF003	Autenticar usuários por e-mail e senha.	Must — protege as funcionalidades.
RF004	Cadastrar e autorizar visitantes antecipadamente.	Must — requisito central da portaria.
RF005	Registrar entrada e saída de visitantes.	Must — mantém histórico de acesso.
RF006	Permitir reserva de áreas comuns.	Must — funcionalidade principal do morador.
RF007	Permitir cancelamento de reservas conforme regras.	Should — complementa o fluxo de reservas.
RF008	Permitir publicação de comunicados pelo síndico.	Must — comunicação oficial.
RF009	Permitir registro de ocorrências por moradores.	Must — canal formal de solicitações.
RF010	Permitir acompanhamento do status de ocorrências.	Must — transparência ao solicitante.
RF011	Cadastrar e gerenciar funcionários.	Should — organização operacional.
RF012	Cadastrar e gerenciar taxas condominiais.	Must — gestão financeira básica.
RF013	Permitir consulta de cobranças e situação financeira.	Must — transparência ao morador.
RF014	Gerar notificações sobre comunicados, reservas e ocorrências.	Should — melhora comunicação.
RF015	Disponibilizar painel administrativo do condomínio.	Should — consolida indicadores.
RF016	Cadastrar veículos vinculados às unidades.	Could — recurso complementar.
RF017	Permitir aprovação ou recusa de reservas pelo síndico.	Should — permite regras específicas.
RF018	Registrar solicitações de manutenção.	Should — organiza atividades de manutenção.
RF019	Permitir acompanhamento das solicitações de manutenção.	Should — informa o andamento.
RF020	Gerar e exportar relatórios administrativos e financeiros.	Could — recurso avançado de gestão.

2.2 Requisitos não funcionais (RNF)

ID	Categoria	Métrica / requisito
RNF001	Desempenho	95% das requisições comuns da API deverão responder em até 300 ms (p95), sob carga de referência definida nos testes.
RNF002	Desempenho	As páginas principais deverão apresentar conteúdo inicial em até 3 s em conexão de banda larga e dispositivo de referência.
RNF003	Disponibilidade	Meta mensal de disponibilidade de 99,5%, excluindo manutenções programadas comunicadas.
RNF004	Escalabilidade	Suportar até 10.000 contas de usuário cadastradas por implantação, com dimensionamento horizontal da camada web/API.
RNF005	Concorrência	Suportar 1.000 sessões autenticadas simultâneas na carga de referência, sem exceder os limites de desempenho.
RNF006	Segurança	Todas as conexões externas deverão utilizar HTTPS com TLS 1.2 ou superior.
RNF007	Segurança	Senhas armazenadas apenas com hash adaptativo BCrypt ou Argon2, nunca em texto puro.
RNF008	Segurança	100% dos endpoints privados deverão validar autenticação e autorização por perfil.
RNF009	Segurança	Após 5 falhas consecutivas de login, aplicar bloqueio temporário de 15 minutos ou desafio equivalente.
RNF010	LGPD	Coletar apenas dados pessoais necessários e registrar finalidade, base legal e prazo de retenção por categoria.
RNF011	LGPD	Acesso a dados pessoais restrito a perfis autorizados; operações sensíveis deverão gerar log de auditoria.
RNF012	LGPD	Solicitações de acesso/correção deverão ser encaminhadas ao responsável e tratadas no prazo legal aplicável.
RNF013	Backup	Executar backup diário e manter retenção mínima de 30 dias, com cópia separada do ambiente principal.
RNF014	Recuperação	Meta de RTO de até 4 horas e RPO de até 24 horas para o banco na primeira versão.

RNF015	Usabilidade	Interface responsiva para larguras a partir de 360 px, sem perda das funções principais.
RNF016	Compatibilidade	Suporte às duas versões estáveis mais recentes de Chrome, Edge e Firefox.
RNF017	Manutenibilidade	Separação em camadas e módulos; build e testes automatizados obrigatórios no pipeline.
RNF018	Acessibilidade	Formulários e fluxos principais deverão atender aos critérios WCAG 2.1 nível AA aplicáveis.

2.3 Histórias de usuário (US)

HU001 — Cadastro de morador

Como administrador, quero cadastrar moradores, para manter os residentes organizados.

Critérios de aceitação (Given/When/Then): Dado que o administrador esteja autenticado, quando preencher os campos obrigatórios e confirmar, então o morador será cadastrado. Se o e-mail já existir, o sistema deverá impedir duplicidade.

Estimativa: 3 pontos.

HU002 — Cadastro de unidade

Como administrador, quero cadastrar unidades, para relacionar moradores às residências.

Critérios de aceitação (Given/When/Then): Dado que a identificação da unidade seja válida, quando salvar, então a unidade será registrada. Se a identificação já existir no mesmo condomínio, o sistema recusará a duplicidade.

Estimativa: 3 pontos.

HU003 — Login

Como usuário, quero entrar com e-mail e senha, para acessar as funções do meu perfil.

Critérios de aceitação (Given/When/Then): Dado que as credenciais sejam válidas, quando enviar o formulário, então o sistema iniciará sessão e emitirá token. Se forem inválidas, retornará erro 401 sem revelar qual campo está incorreto.

Estimativa: 5 pontos.

HU004 — Autorizar visitante

Como morador, quero autorizar visitante antecipadamente, para agilizar a entrada.

Critérios de aceitação (Given/When/Then): Dado que o morador esteja autenticado e vinculado à unidade, quando informar visitante e período válido, então a autorização será registrada. A portaria deverá conseguir consultá-la durante a validade.

Estimativa: 5 pontos.

HU005 — Registrar acesso

Como porteiro, quero registrar entrada e saída, para manter o histórico de visitantes.

Critérios de aceitação (Given/When/Then): Dado que exista autorização válida, quando registrar entrada, então o sistema armazenará horário e operador. Ao registrar saída, atualizará o mesmo registro.

Estimativa: 5 pontos.

HU006 — Reservar área comum

Como morador, quero reservar uma área comum, para utilizar o espaço em um período disponível.

Critérios de aceitação (Given/When/Then): Dado que a área e o período estejam disponíveis e as regras sejam atendidas, quando confirmar, então a reserva será registrada. Conflito deverá ser impedido.

Estimativa: 8 pontos.

HU007 — Cancelar reserva

Como morador, quero cancelar uma reserva minha, para liberar o período.

Critérios de aceitação (Given/When/Then): Dado que a reserva pertença ao morador e esteja dentro do prazo de cancelamento, quando solicitar cancelamento, então o status será atualizado. Reserva já cancelada não poderá ser cancelada novamente.

Estimativa: 3 pontos.

HU008 — Publicar comunicado

Como síndico, quero publicar avisos, para informar os moradores.

Critérios de aceitação (Given/When/Then): Dado que o síndico tenha permissão, quando publicar título e conteúdo válidos, então o comunicado ficará visível aos moradores do condomínio.

Estimativa: 3 pontos.

HU009 — Registrar ocorrência

Como morador, quero registrar uma ocorrência, para comunicar problemas.

Critérios de aceitação (Given/When/Then): Dado que o morador esteja autenticado, quando enviar descrição válida, então o sistema criará ocorrência com identificador e status inicial ABERTA.

Estimativa: 5 pontos.

HU010 — Acompanhar ocorrência

Como morador, quero acompanhar ocorrência, para saber seu andamento.

Critérios de aceitação (Given/When/Then): Dado que exista ocorrência vinculada ao morador, quando consultar o histórico, então visualizará status e atualizações autorizadas.

Estimativa: 3 pontos.

HU011 — Gerenciar funcionários

Como síndico, quero cadastrar e atualizar funcionários, para manter a equipe organizada.

Critérios de aceitação (Given/When/Then): Dado que o síndico tenha permissão, quando cadastrar dados obrigatórios, então o funcionário será registrado. Alterações posteriores deverão ser auditadas.

Estimativa: 5 pontos.

HU012 — Gerenciar cobrança

Como síndico, quero cadastrar cobranças, para organizar taxas por unidade.

Critérios de aceitação (Given/When/Then): Dado que a unidade exista e valor/vencimento sejam válidos, quando confirmar, então a cobrança será vinculada à unidade e ficará disponível para consulta.

Estimativa: 8 pontos.

HU013 — Consultar financeiro

Como morador, quero consultar cobranças, para acompanhar minha situação financeira.

Critérios de aceitação (Given/When/Then): Dado que esteja autenticado, quando abrir a área financeira, então verá apenas cobranças de unidades às quais está vinculado, com valor, vencimento e status.

Estimativa: 5 pontos.

HU014 — Notificações

Como usuário, quero receber notificações, para acompanhar atualizações importantes.

Critérios de aceitação (Given/When/Then): Dado que ocorra evento relevante, quando comunicado, reserva ou ocorrência for atualizado, então o sistema criará notificação para os usuários destinatários.

Estimativa: 5 pontos.

HU015 — Relatórios

Como síndico, quero gerar relatórios, para acompanhar a gestão.

Critérios de aceitação (Given/When/Then): Dado que existam dados e o síndico tenha permissão, quando solicitar relatório por período, então o sistema apresentará os resultados e permitirá exportação autorizada.

Estimativa: 8 pontos.

Estimativa total: 74 story points (5 histórias de 3 pontos, 7 histórias de 5 pontos e 3 histórias de 8 pontos (74 pontos no total)). A escala é relativa e deverá ser recalibrada após o primeiro sprint.

BLOCO 3 — ARQUITETURA DE SOFTWARE

3.1 Visão arquitetural

O CondoSmart adotará arquitetura cliente-servidor com backend em monólito modular. Os módulos compartilham uma aplicação Spring Boot e um banco PostgreSQL, mas mantêm responsabilidades separadas. Essa escolha reduz a complexidade operacional inicial em comparação com microsserviços, sem impedir uma extração futura caso haja necessidade comprovada.

3.2 Registros de decisões arquiteturais (ADR)

ADR-001 — Monólito modular

O domínio inicial possui módulos relacionados e uma equipe pequena.
Decisão: monólito modular.
Alternativas: microsserviços (mais complexidade de rede, observabilidade e dados distribuídos) e monólito sem módulos (maior acoplamento).
Consequências: implantação mais simples, transações locais e necessidade de manter fronteiras internas claras.

ADR-002 — Backend Java / Spring Boot

Requisitos incluem API REST, autenticação, regras transacionais e integração PostgreSQL.
Decisão: Java 21 + Spring Boot.
Alternativas: Node/Express (menor estrutura integrada de segurança e persistência) e Django (ecossistema Python diferente da stack adotada).
Consequências: uso de Spring Security, Spring Data JPA, validação e testes com JUnit.

ADR-003 — Frontend React

A interface deve ser responsiva, modular e reutilizável.
Decisão: React com TypeScript.
Alternativas: Angular (framework mais opinativo e abrangente) e Vue (válido, mas não escolhido para padronização da stack).
Consequências: componentes reutilizáveis e tipagem no frontend.

ADR-004 — PostgreSQL relacional

O sistema possui relações fortes entre unidades, moradores, reservas, visitantes e cobranças.
Decisão: PostgreSQL.
Alternativas: MySQL (relacional e viável, mas sem vantagem decisiva para este modelo) e MongoDB (flexibilidade documental, mas maior esforço para integridade referencial e transações relacionais).
Consequências: PK, FK, constraints, transações e normalização até 3FN.

ADR-005 — REST/JSON

A interface web precisa de operações CRUD e consulta simples.
Decisão: API REST com JSON sobre HTTPS.

Alternativas: GraphQL (flexibilidade maior, porém complexidade de autorização e consultas) e gRPC (adequado a comunicação serviço-serviço, mas menos direto para navegador).
Consequências: endpoints HTTP, códigos de status e contrato OpenAPI 3.0.

ADR-006 — Segurança

Há cinco perfis com permissões distintas.
Decisão: Spring Security, JWT de curta duração e RBAC.
Alternativas: sessão centralizada (viável, mas requer gerenciamento de sessão) e controle apenas no frontend (inseguro).
Consequências: autorização no backend em cada endpoint, rotação/expiração de tokens e trilha de auditoria.

3.3 Stack tecnológica comparada

Camada	Escolha	Alternativa descartada	Motivo técnico
Frontend	React + TypeScript	Angular / Vue	Componentização e tipagem; suficiente para interface web responsiva.
Backend	Java 21 + Spring Boot	Node.js/Express; Django	Suporte integrado a API, validação, segurança, transações e persistência.
Persistência	PostgreSQL	MySQL / MongoDB	Modelo relacional, constraints e integridade entre entidades.
Comunicação	REST + JSON / HTTPS	GraphQL / gRPC	Compatibilidade direta com navegador e operações CRUD.
Segurança	Spring Security + JWT + RBAC	Autorização apenas no cliente	Validação no servidor e perfis centralizados.
Documentação	OpenAPI 3.0 / Swagger	Documentação manual isolada	Contrato legível e testável.
DevOps	Docker + GitHub Actions	Deploy manual	Ambientes repetíveis e pipeline automatizado.
Migrações	Flyway	Scripts manuais sem versão	Histórico de alterações e execução controlada.

3.4 Modelo C4 — quatro níveis

Nível 1 — Contexto

O CondoSmart é utilizado por morador, síndico, porteiro, funcionário e administrador. Cada ator acessa funções compatíveis com seu perfil.

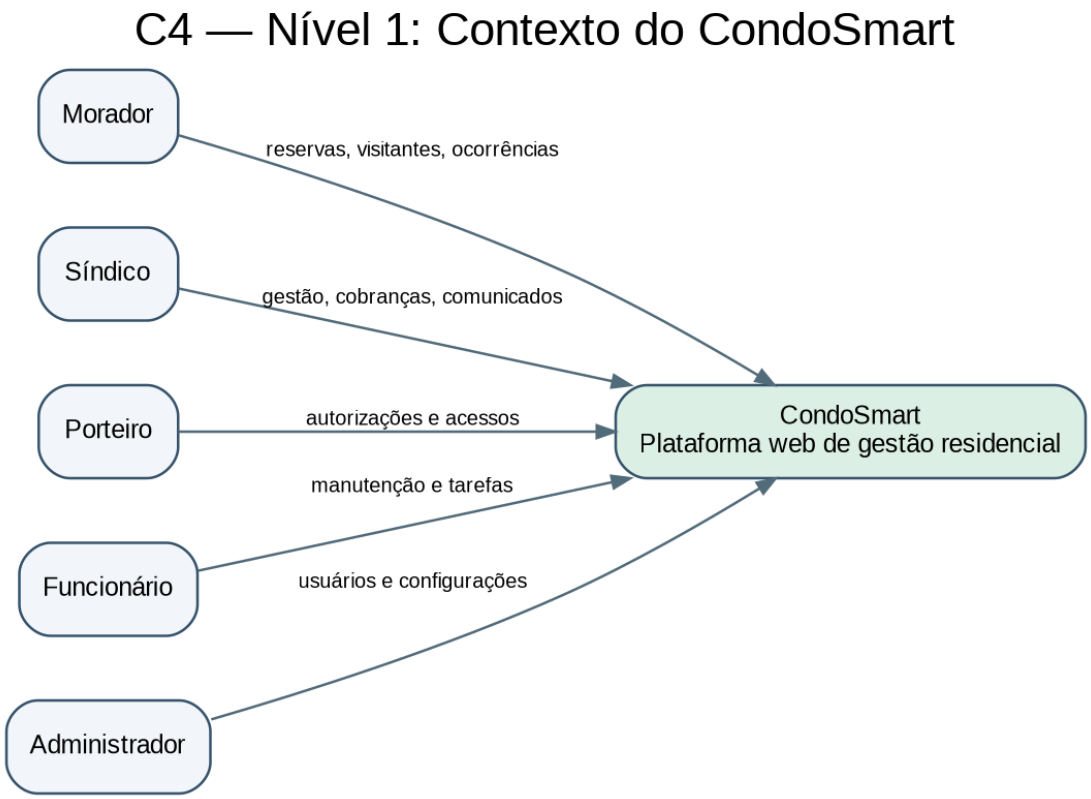


Figura 1 — C4 contexto. Autores: Leandro Estrela, Luis Henrique. Data: 08/11/2026.

Nível 2 — Containers

Os containers são: frontend React servido por Nginx; backend Java/Spring Boot expondo API REST; PostgreSQL em rede privada; armazenamento de objetos para backups e anexos. Comunicação: HTTPS 443, API JSON e conexão PostgreSQL 5432 protegida.

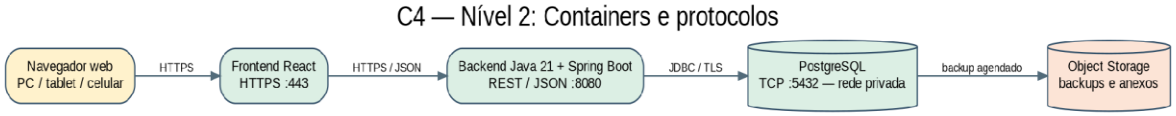


Figura 2 — C4 containers. Autores: Leandro Estrela, Luis Henrique. Data: 08/11/2026.

Nível 3 — Componentes

Os componentes do backend incluem autenticação, usuários/unidades, visitantes/acessos, reservas, ocorrências, financeiro, comunicados/notificações, camada de serviços e repositórios.

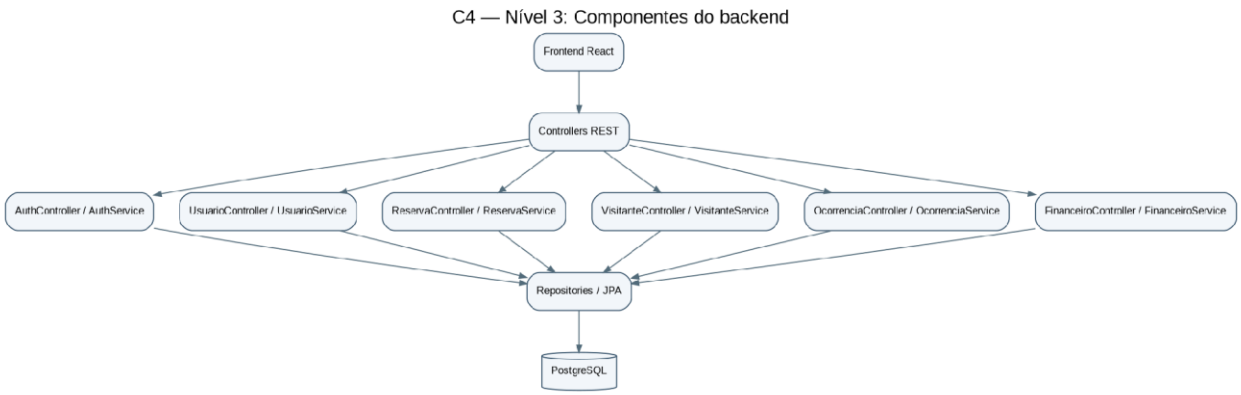


Figura 3 — C4 componentes. Autores: Leandro Estrela, Luis Henrique. Data: 08/11/2026.

Nível 4 — Código de componente crítico

Componente escolhido: ReservaService. Ele verifica conflito de horário, aplica regras de reserva e persiste o registro em transação. A restrição de banco e o controle transacional devem impedir duas reservas simultâneas conflitantes.

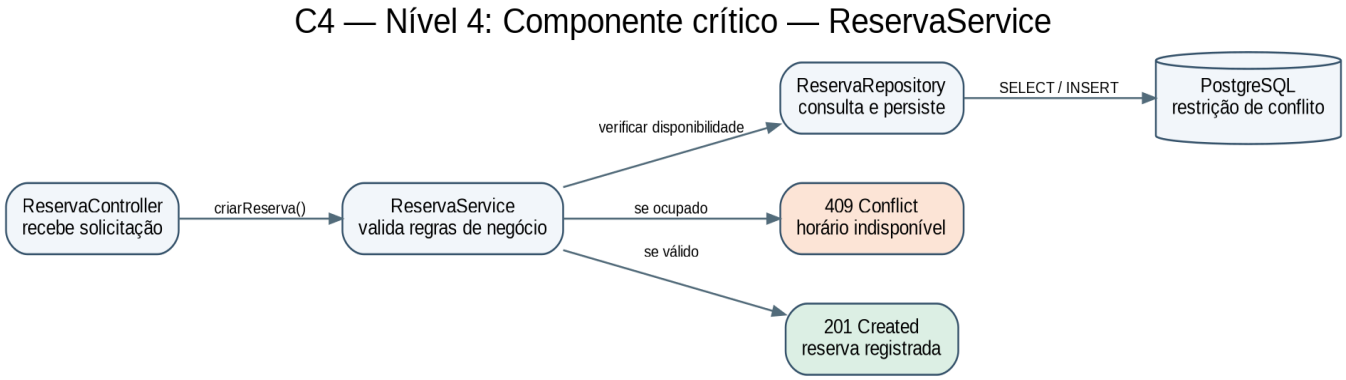


Figura 4 — C4 código/componente crítico. Autores:Leandro Estrela, Luis Henrique. Data: 08/11/2026.

3.5 Padrões de projeto

Padrão	Aplicação	Justificativa
Repository	Acesso às entidades persistidas.	Isola SQL/persistência das regras de negócio.
Service Layer	Regras de reserva, acesso, cobranças e ocorrências.	Centraliza lógica e facilita testes.
DTO	Entrada e saída dos endpoints.	Evita expor entidades e controla dados transferidos.

Dependency Injection	Serviços e repositórios Spring.	Reduz acoplamento e facilita substituição em testes.
Strategy	Políticas variáveis de autorização e regras de reserva.	Permite adicionar políticas sem condicional monolítico.
Factory	Criação padronizada de notificações por tipo de evento.	Centraliza construção e seleção do tipo de notificação.

BLOCO 4 — MODELAGEM UML

A modelagem utiliza UML para descrever atores, estrutura estática, interações, componentes, implantação e processos. Todos os diagramas devem ser apresentados e explicados oralmente, conforme orientação do professor.

4.1 Diagrama de Caso de Uso

Atores: Morador, Síndico, Porteiro, Funcionário e Administrador. Os casos de uso cobrem login, autorização de visitantes, reservas, ocorrências, consulta financeira, comunicados, gestão de funcionários, registro de acesso, manutenção, usuários/unidades e relatórios. A autenticação é uma precondição comum; a autorização de perfil é verificada no backend.

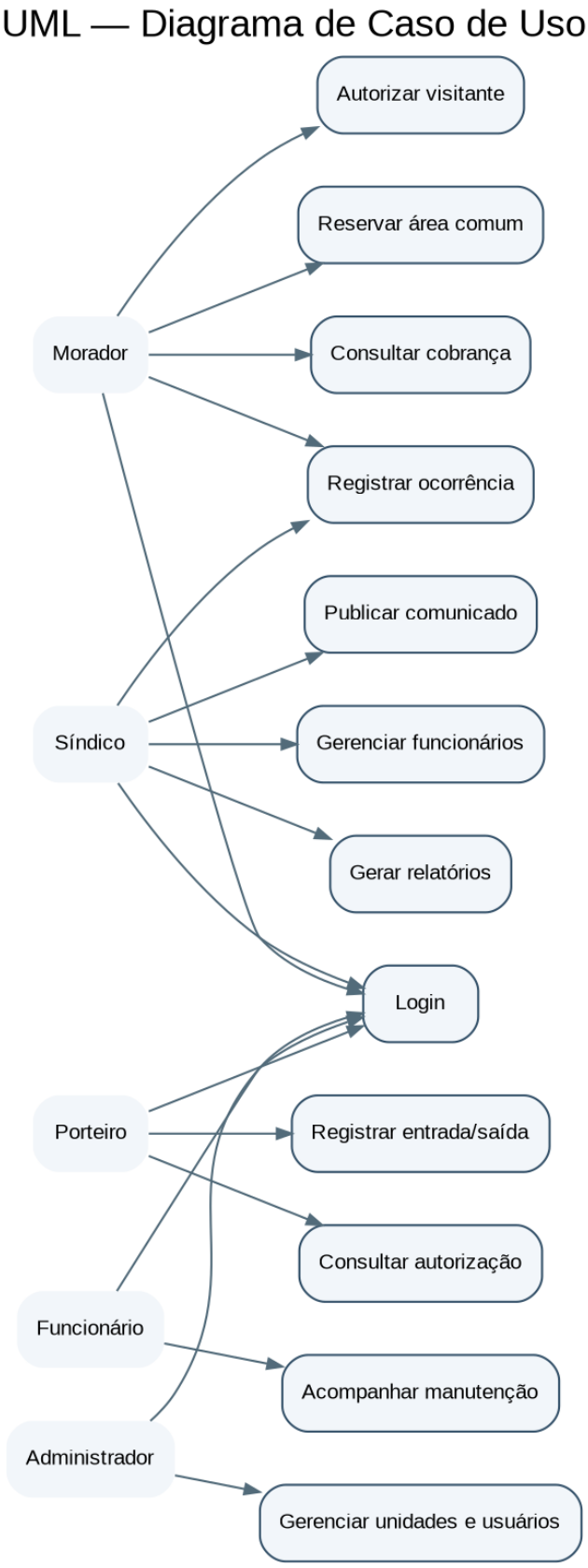


Figura 5 — UML caso de uso. Autores: Leandro Estrela, Luis Henrique. Data: 08/11/2026.

4.2 Diagrama de Classes

O modelo apresenta 13 classes principais, com atributos tipados, visibilidade (- privado, + público), métodos e relações. Morador e Funcionário especializam Usuario. O perfil de síndico, porteiro e administrador é representado por enumeração de perfil e permissões RBAC, não por subclasses duplicadas.

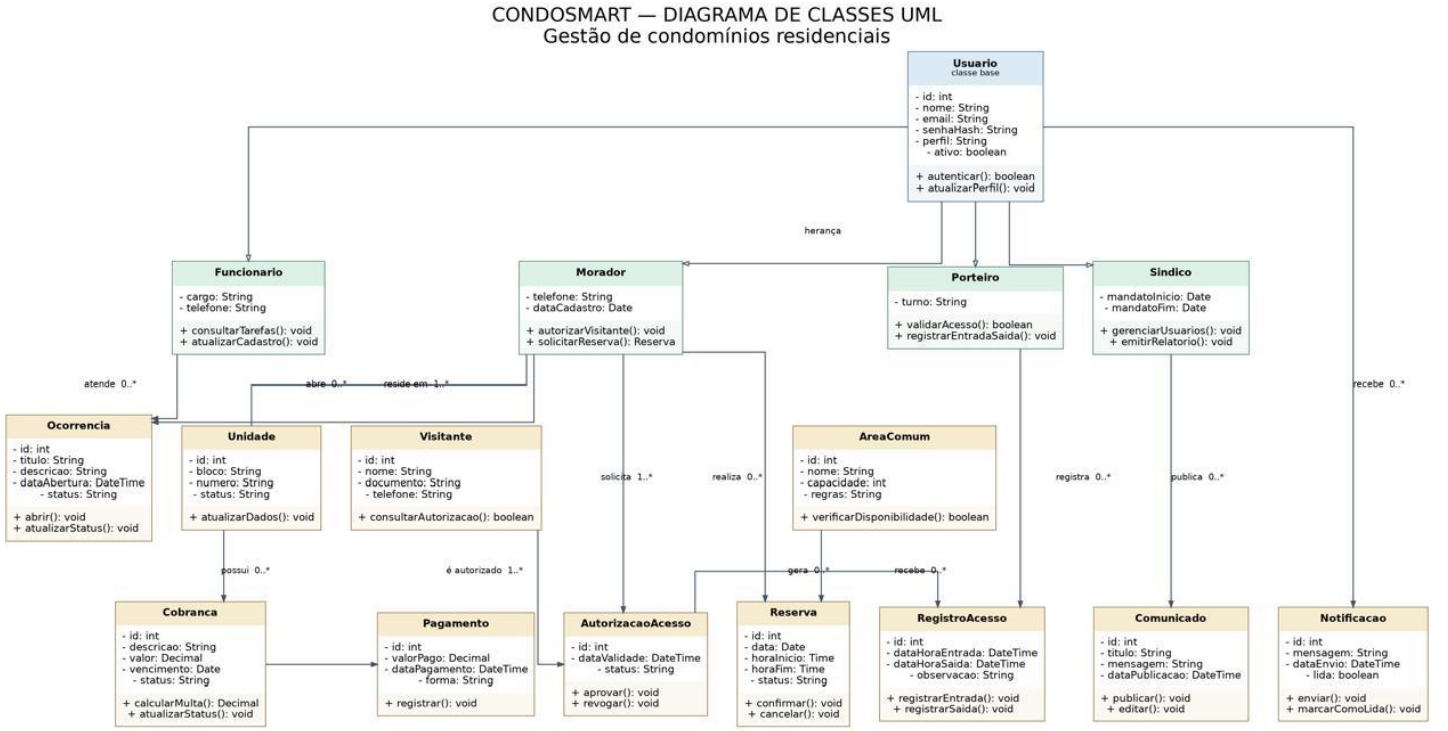


Figura 6 — UML classes. Autores: Leandro Estrela, Luis Henrique. Data: 08/11/2026.

4.3 Diagramas de Sequência

Autenticação

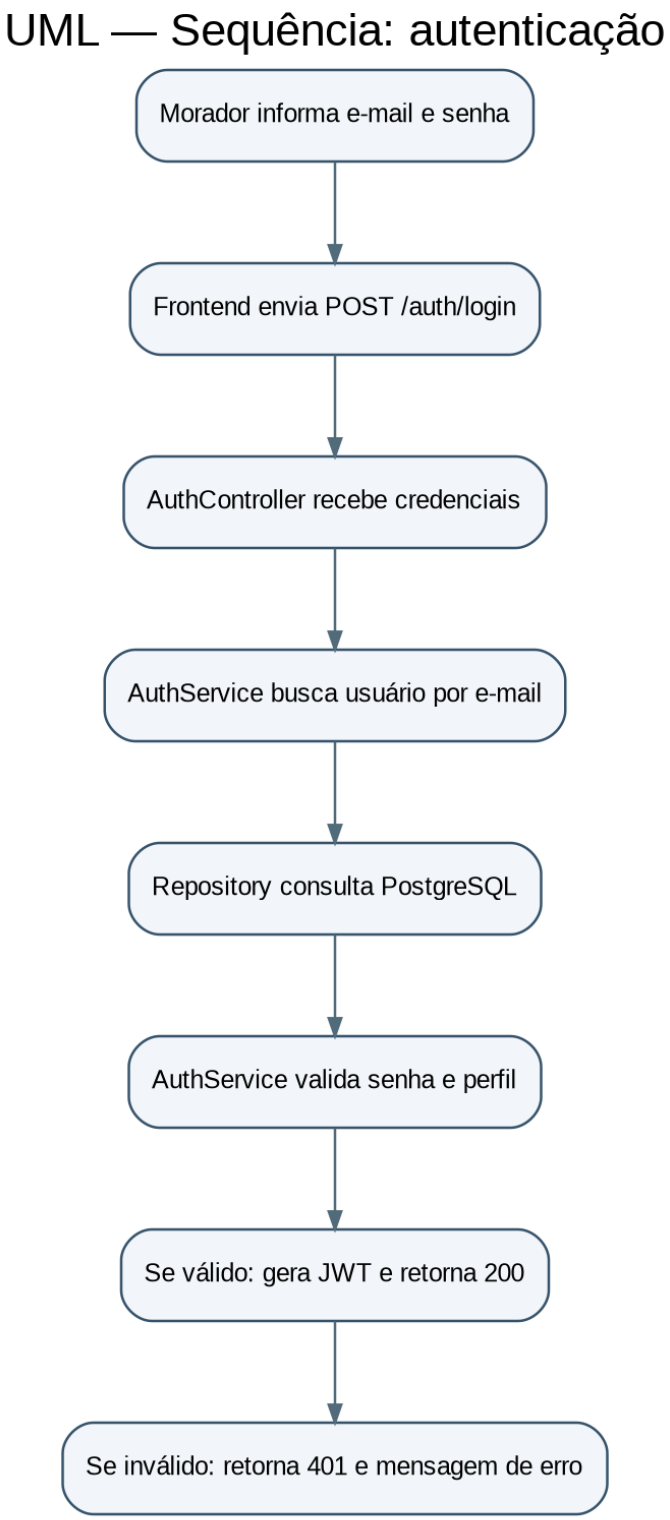


Figura 7 — Sequência de autenticação. Autores: Leandro Estrela, Luis Henrique. Data: 08/11/2026.

Fluxo principal — reserva bem-sucedida

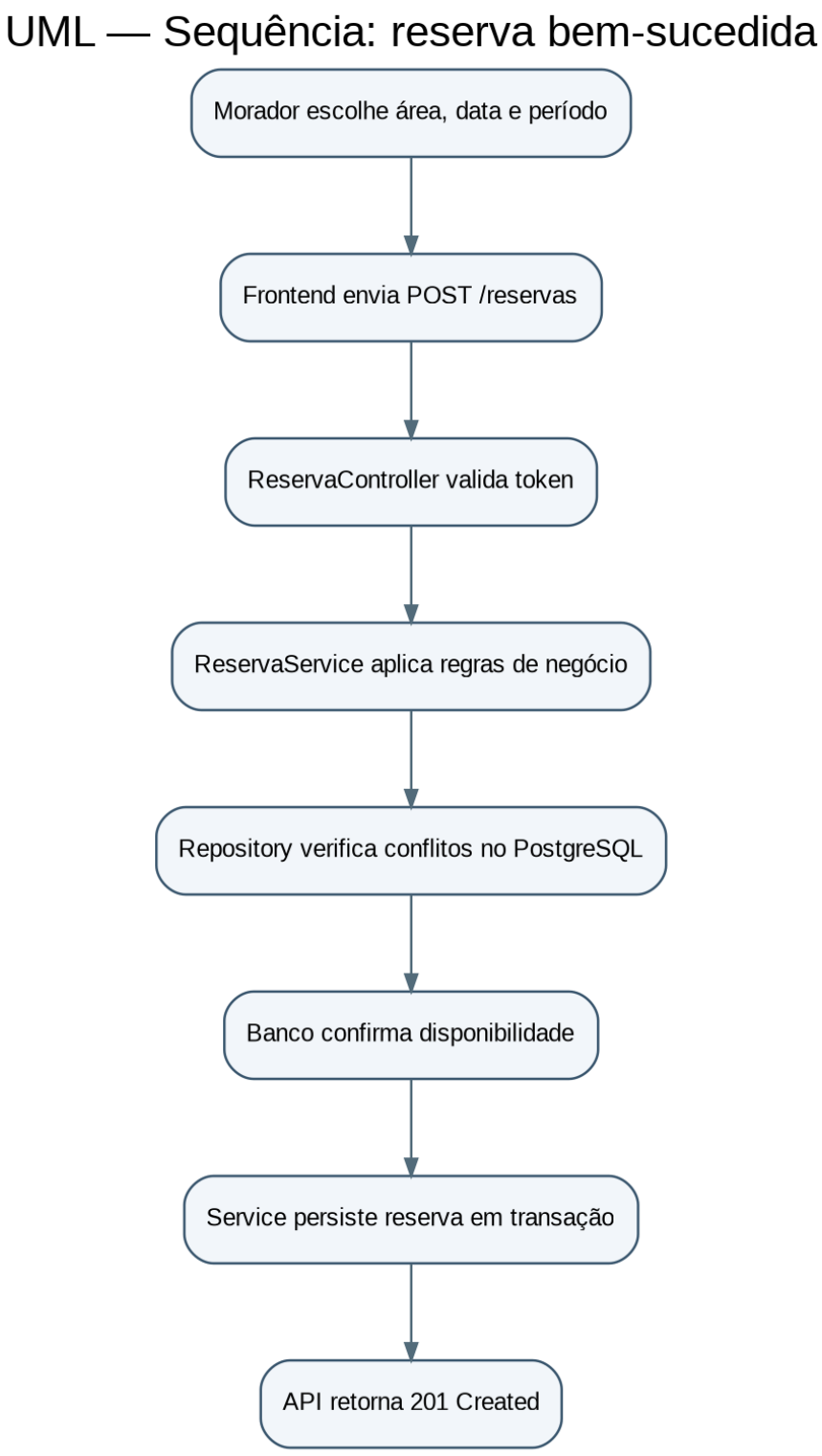


Figura 8 — Sequência de reserva. Autores: Leandro Estrela, Luis Henrique. Data: 08/11/2026.

Erro crítico — conflito de reserva

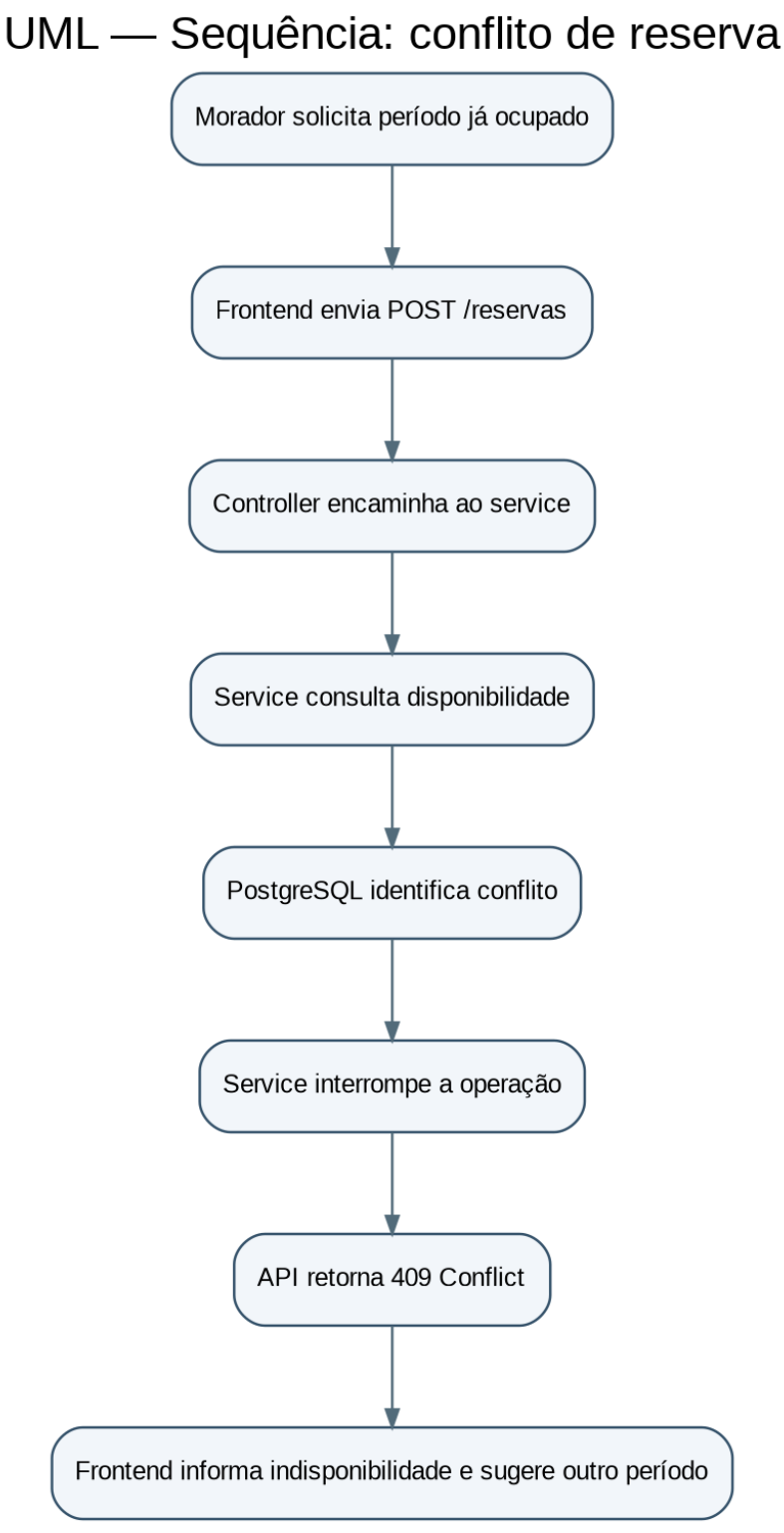


Figura 9 — Sequência de erro crítico. Autores: Leandro Estrela, Luis Henrique. Data: 08/11/2026.

4.4 Diagrama de Componentes

Apresenta frontend React, interface REST, módulos internos do backend, persistência JPA/PostgreSQL e armazenamento externo de backups. As dependências partem da interface para os módulos de negócio e destes para a camada de persistência.

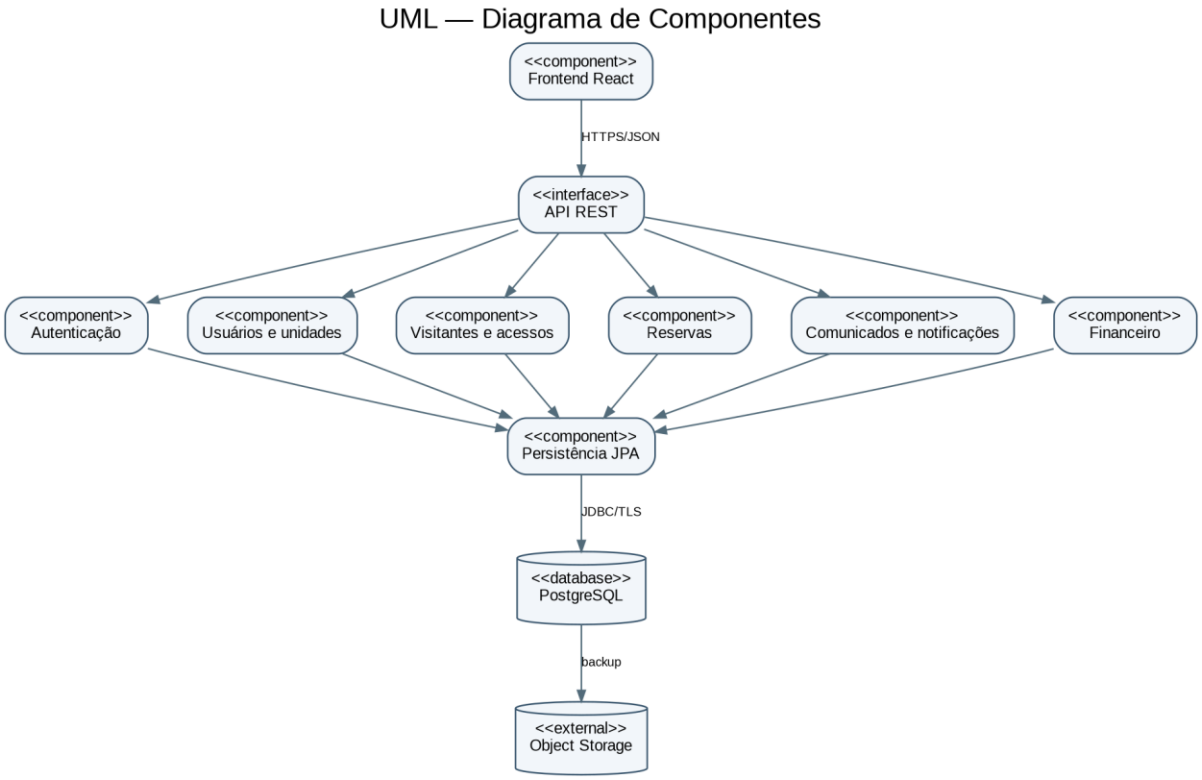


Figura 10 — UML componentes. Autores: Leandro Estrela, Luis Henrique. Data: 08/11/2026.

4.5 Diagrama de Implantação

A implantação considera ambientes DEV, STAGING e PROD. O tráfego externo utiliza HTTPS 443; o balanceador distribui para servidores web; a API utiliza porta interna 8080; o banco PostgreSQL utiliza 5432 somente na rede privada. A arquitetura de produção prevê instâncias redundantes e banco gerenciado HA.

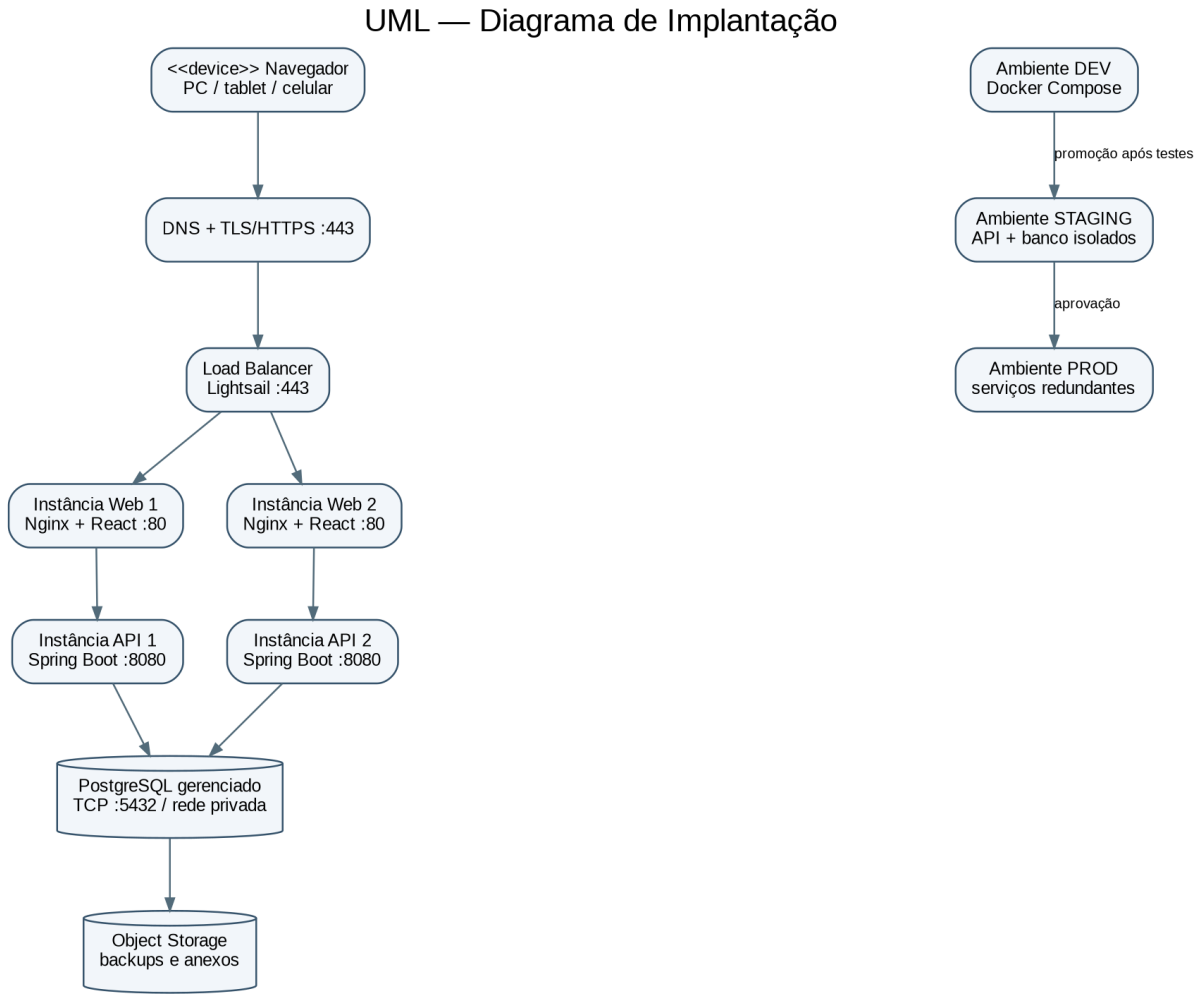


Figura 11 — UML deployment. Autores: Leandro Estrela, Luis Henrique. Data: 08/11/2026.

4.6 Diagramas de Atividades

Processo principal — reserva de área comum

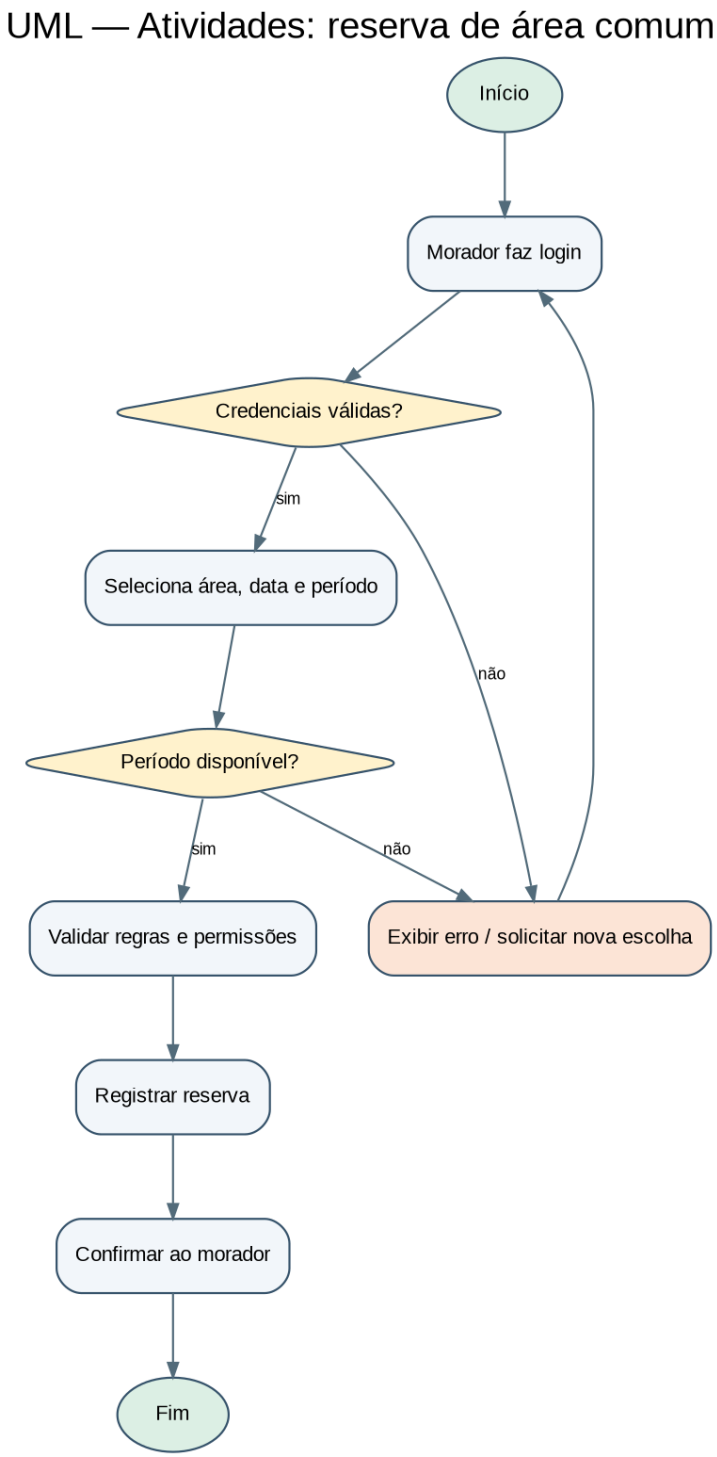


Figura 12 — Atividades: reserva. Autores: Leandro Estrela, Luis Henrique. Data: 08/11/2026.

Processo administrativo — gerenciamento de cobrança

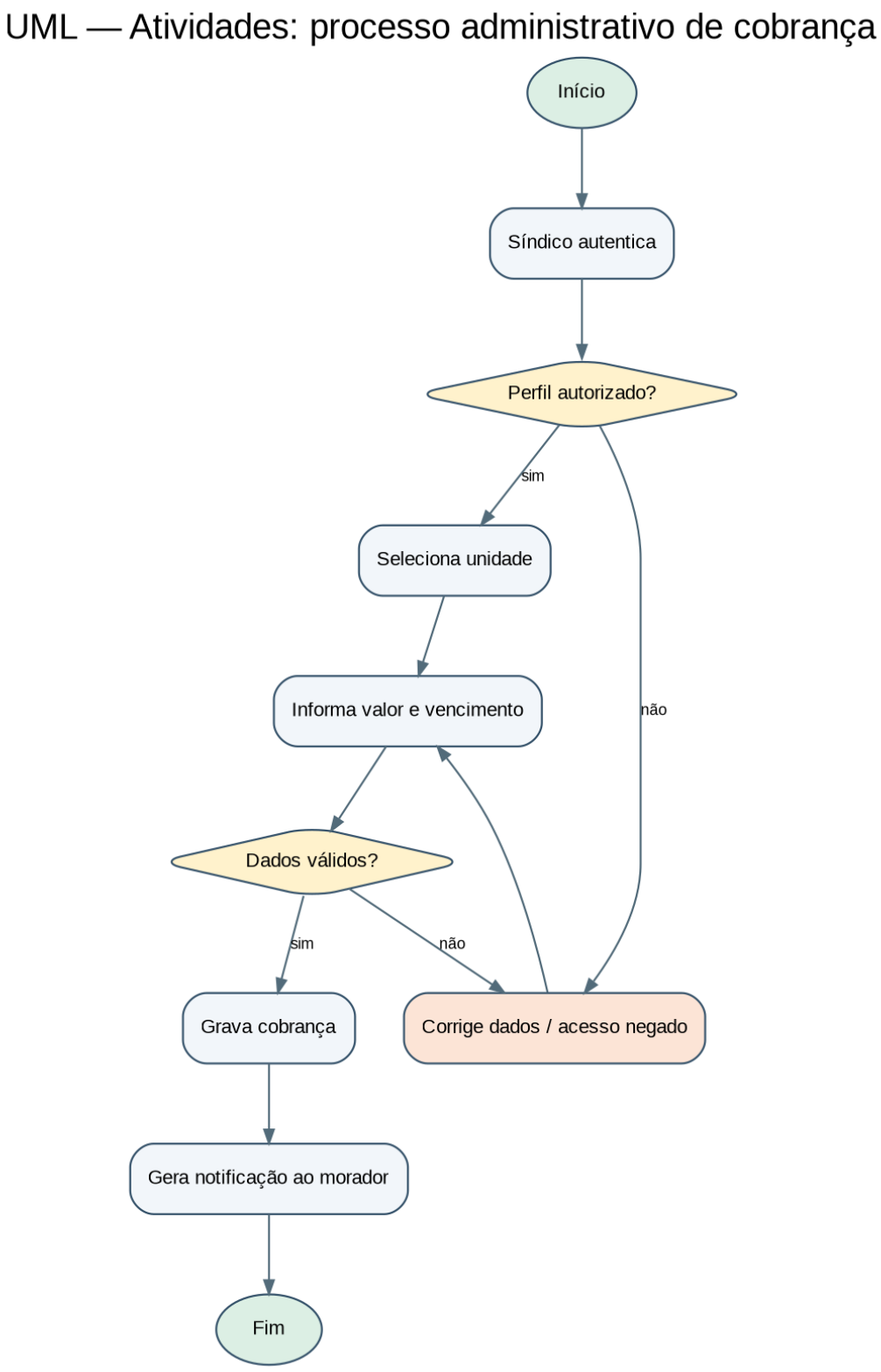


Figura 13 — Atividades: cobrança. Autores:Leandro Estrela, Luis Henrique. Data: 08/11/2026.

BLOCO 5 — BANCO DE DADOS

5.1 Decisão do SGBD

Banco	Vantagens	Alternativa / motivo da decisão
PostgreSQL — escolhido	Relacional, transações ACID, constraints, índices e integridade referencial.	Atende ao domínio com relações entre moradores, unidades, reservas e cobranças.
MySQL	SGBD relacional maduro e amplamente utilizado.	Também seria viável; PostgreSQL foi escolhido como padrão técnico do projeto.
MongoDB	Modelo documental flexível.	Não foi priorizado porque o domínio exige relações, integridade referencial e transações entre registros.

5.2 Modelo conceitual ER

Entidades: Usuário, Unidade, Morador, Funcionário, Visitante, AutorizaçãoVisitante, RegistroAcesso, ÁreaComum, Reserva, Comunicado, Ocorrência, Cobrança e Notificação. As cardinalidades estão representadas na figura.

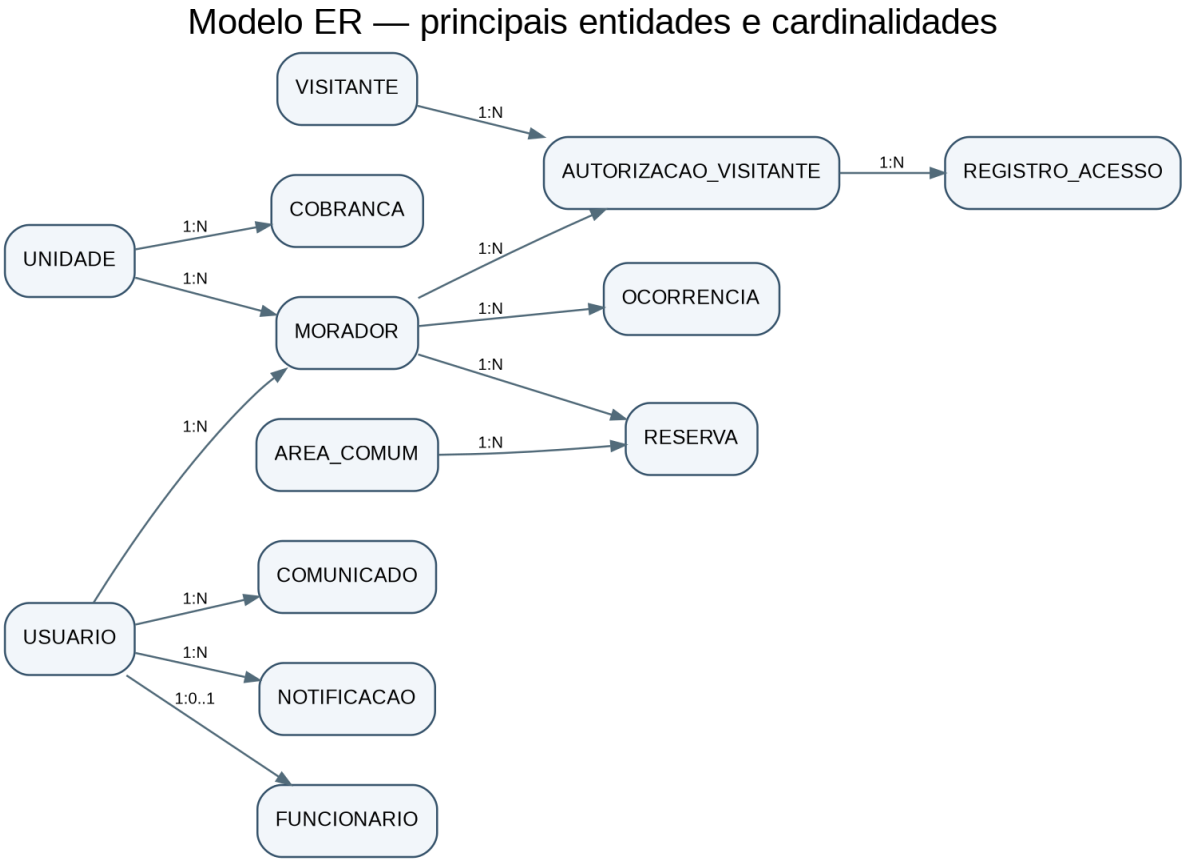


Figura 14 — Modelo ER conceitual. Autores: Leandro Estrela, Luis Henrique. Data: 08/11/2026.

5.3 Modelo lógico

Tabela	Atributos principais	Chaves / índices
usuario	id BIGINT, nome VARCHAR(120), email VARCHAR(150), senha_hash VARCHAR(255), perfil VARCHAR(20), ativo BOOLEAN	PK id; UNIQUE email; índice de busca por email.
unidade	id BIGINT, identificacao VARCHAR(30), bloco VARCHAR(30), andar INTEGER	PK id; UNIQUE (identificacao, bloco).
morador	id BIGINT, usuario_id BIGINT, unidade_id BIGINT, telefone VARCHAR(20)	PK id; FKs usuario/unidade; UNIQUE (usuario_id, unidade_id).
funcionario	id BIGINT, usuario_id BIGINT, cargo VARCHAR(80), telefone VARCHAR(20)	PK id; FK usuario; UNIQUE usuario_id.
visitante	id BIGINT, nome VARCHAR(120), documento VARCHAR(30)	PK id; documento com acesso restrito.
autorizacao_visitante	id BIGINT, morador_id BIGINT, visitante_id BIGINT, inicio/fim TIMESTAMP, status	PK id; FKs morador/visitante; índice por período/status.
registro_acesso	id BIGINT, autorizacao_id BIGINT, entrada/saida TIMESTAMP	PK id; FK autorização; índice por entrada.
area_comum	id BIGINT, nome VARCHAR(100), capacidade INTEGER, ativa BOOLEAN	PK id; CHECK capacidade > 0.
reserva	id BIGINT, morador_id BIGINT, area_id BIGINT, inicio/fim TIMESTAMP, status	PK id; FKs; índice por área/período; restrição de não sobreposição.
comunicado	id BIGINT, autor_id BIGINT, titulo VARCHAR(150), conteudo TEXT, data_publicacao TIMESTAMP	PK id; FK usuário; índice por data.
ocorrencia	id BIGINT, morador_id BIGINT, descricao TEXT, status VARCHAR(30), data_registro TIMESTAMP	PK id; FK morador; índice por status.
cobranca	id BIGINT, unidade_id BIGINT, valor NUMERIC(10,2), vencimento DATE, status VARCHAR(20)	PK id; FK unidade; índice unidade/vencimento.
notificacao	id BIGINT, usuario_id BIGINT, mensagem TEXT, data_envio TIMESTAMP, lida BOOLEAN	PK id; FK usuário; índice usuário/lida.

5.4 Modelo físico — DDL executável

O script abaixo cria as tabelas principais no PostgreSQL. O modelo usa intervalos de início e fim para reservas. A exclusão de sobreposição deve ser garantida por uma constraint EXCLUDE usando btree_gist, além da validação transacional na aplicação.

```
CREATE EXTENSION IF NOT EXISTS btree_gist;

CREATE TABLE usuario (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nome VARCHAR(120) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  senha_hash VARCHAR(255) NOT NULL,
  perfil VARCHAR(20) NOT NULL CHECK
    (perfil IN ('MORADOR','SINDICO','PORTEIRO','FUNCIONARIO','ADMIN')),
  ativo BOOLEAN NOT NULL DEFAULT TRUE
);
```

```
CREATE TABLE unidade (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  identificacao VARCHAR(30) NOT NULL,
  bloco VARCHAR(30) NOT NULL DEFAULT '',
  andar INTEGER,
  UNIQUE (identificacao, bloco)
);

CREATE TABLE morador (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  usuario_id BIGINT NOT NULL REFERENCES usuario(id),
  unidade_id BIGINT NOT NULL REFERENCES unidade(id),
  telefone VARCHAR(20),
  UNIQUE (usuario_id, unidade_id)
);

CREATE TABLE funcionario (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  usuario_id BIGINT NOT NULL UNIQUE REFERENCES usuario(id),
  cargo VARCHAR(80) NOT NULL,
  telefone VARCHAR(20)
);

CREATE TABLE visitante (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nome VARCHAR(120) NOT NULL,
  documento_criptografado TEXT NOT NULL
);

CREATE TABLE autorizacao_visitante (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  morador_id BIGINT NOT NULL REFERENCES morador(id),
  visitante_id BIGINT NOT NULL REFERENCES visitante(id),
  inicio TIMESTAMP NOT NULL,
  fim TIMESTAMP NOT NULL,
  status VARCHAR(20) NOT NULL,
  CHECK (fim > inicio)
);

CREATE TABLE registro_acesso (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  autorizacao_id BIGINT NOT NULL REFERENCES autorizacao_visitante(id),
  entrada TIMESTAMP NOT NULL,
  saida TIMESTAMP,
  CHECK (saida IS NULL OR saida >= entrada)
);

CREATE TABLE area_comum (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nome VARCHAR(100) NOT NULL,
  capacidade INTEGER NOT NULL CHECK (capacidade > 0),
  ativa BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE reserva (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  morador_id BIGINT NOT NULL REFERENCES morador(id),
  area_id BIGINT NOT NULL REFERENCES area_comum(id),
  periodo TSTZRANGE NOT NULL,
  status VARCHAR(20) NOT NULL,
  CHECK (NOT isempty(periodo)),
  EXCLUDE USING gist (
    area_id WITH =,
    periodo WITH &&
  ) WHERE (status IN ('PENDENTE','CONFIRMADA'))
);

CREATE TABLE comunicado (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  autor_id BIGINT NOT NULL REFERENCES usuario(id),
  titulo VARCHAR(150) NOT NULL,
  conteudo TEXT NOT NULL,
  data_publicacao TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
```

```
);

CREATE TABLE ocorrencia (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  morador_id BIGINT NOT NULL REFERENCES morador(id),
  descricao TEXT NOT NULL,
  status VARCHAR(30) NOT NULL DEFAULT 'ABERTA',
  data_registro TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE cobranca (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  unidade_id BIGINT NOT NULL REFERENCES unidade(id),
  valor NUMERIC(10,2) NOT NULL CHECK (valor >= 0),
  vencimento DATE NOT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'PENDENTE'
);

CREATE TABLE notificacao (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  usuario_id BIGINT NOT NULL REFERENCES usuario(id),
  mensagem TEXT NOT NULL,
  data_envio TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  lida BOOLEAN NOT NULL DEFAULT FALSE
);

CREATE INDEX idx_morador_unidade ON morador(unidade_id);
CREATE INDEX idx_autorizacao_periodo ON autorizacao_visitante(inicio, fim);
CREATE INDEX idx_registro_entrada ON registro_acesso(entrada);
CREATE INDEX idx_ocorrencia_status ON ocorrencia(status);
CREATE INDEX idx_cobranca_unidade_vencimento ON cobranca(unidade_id, vencimento);
CREATE INDEX idx_notificacao_usuario_lida ON notificacao(usuario_id, lida);
```

5.5 Dicionário de dados e classificação LGPD

Campo	Tipo	Restrição / classificação	Descrição
usuario.nome	VARCHAR(120)	NOT NULL; dado pessoal	Nome do usuário.
usuario.email	VARCHAR(150)	UNIQUE; dado pessoal	Identificador de acesso e contato.
usuario.senha_hash	VARCHAR(255)	NOT NULL; credencial	Hash adaptativo da senha.
visitante.documento_criptografado	TEXT	Dado pessoal; criptografia	Documento do visitante protegido em repouso.
morador.telefone	VARCHAR(20)	Dado pessoal	Contato do morador.
reserva.periodo	TSTZRANGE	NOT NULL	Intervalo de utilização da área comum.
cobranca.valor	NUMERIC(10,2)	CHECK >= 0; dado financeiro	Valor da cobrança da unidade.
registro_acesso.entrada	TIMESTAMP	Dado pessoal / histórico	Horário de entrada do visitante.
notificacao.lida	BOOLEAN	NOT NULL	Indica se o usuário visualizou a notificação.

5.6 Backup, retenção e recuperação

- Backup diário automatizado, retenção mínima de 30 dias e armazenamento criptografado em conta/serviço separado.

- RPO inicial de até 24 horas e RTO de até 4 horas; metas sujeitas a testes de restauração.
- Teste mensal de restauração em ambiente isolado e registro do resultado.
- Acesso aos backups restrito a administradores autorizados; logs de acesso e cópias protegidas.

5.7 LGPD e migrações sem downtime

O tratamento deverá possuir finalidade e base legal documentadas, minimização de dados, controle de acesso, canal para exercício de direitos e política de retenção. Dados de visitantes e registros de acesso terão retenção limitada e justificada. O documento não define prazo legal único para todos os registros: a política deverá ser validada com o controlador e a assessoria jurídica.

As migrações serão versionadas pelo Flyway e seguirão expand-and-contract: adicionar campos compatíveis, publicar versão que aceite esquema antigo e novo, migrar dados em lotes, validar integridade, trocar leituras/escritas e remover campos antigos apenas após janela de segurança.

BLOCO 6 — INFRAESTRUTURA E DEVOPS (BLOCO OPCIONAL)

6.1 Topologia de infraestrutura

A proposta de produção utiliza a região AWS São Paulo (sa-east-1), rede virtual gerenciada pelo provedor, sub-rede pública para balanceador e camada web, e sub-rede privada para API e PostgreSQL. O banco não terá endpoint público. A topologia utiliza um balanceador e duas instâncias de aplicação para reduzir o ponto único de falha.

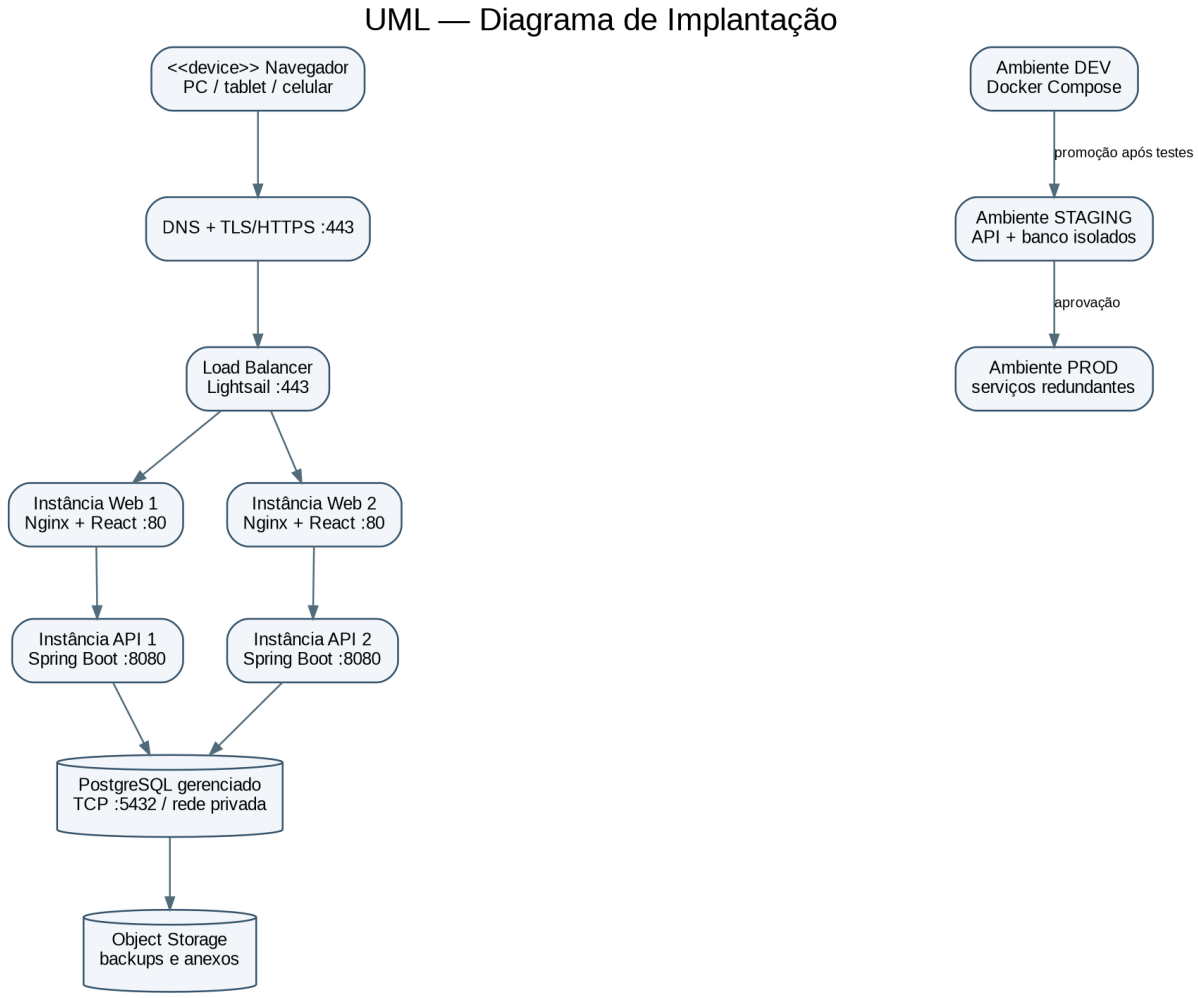


Figura 15 — Implantação por ambiente. Autores: Leandro Estrela, Luis Henrique. Data: 08/11/2026.

6.2 Estimativa mensal por ambiente

Ambiente	Itens considerados	Total mensal (USD)
DEV	1 instância Linux 2 GB — US$ 12; banco em Docker local; recursos compartilhados de equipe.	12
STAGING	1 instância Linux 4 GB — US$ 24; banco gerenciado 1 GB padrão — US$ 15; bucket 5 GB — US$ 1.	40

PROD	2 instâncias Linux 4 GB — US$ 48; balanceador — US$ 18; PostgreSQL gerenciado HA 4 GB — US$ 120; bucket 100 GB — US$ 3.	189
TOTAL	Soma dos três ambientes ativos.	241

6.3 Rede e segurança

- VPC/rede virtual isolada por ambiente, com segmentação lógica de sub-redes públicas e privadas.
- Load balancer recebe HTTPS 443 e distribui para instâncias web saudáveis.
- API recebe tráfego somente das camadas autorizadas; porta 8080 não exposta publicamente.
- PostgreSQL recebe conexões TCP 5432 somente das instâncias de aplicação.
- Acesso administrativo por SSH restrito a IPs autorizados e chaves; sem senha compartilhada.
- Segredos em serviço de gerenciamento de segredos; nunca versionar .env ou chaves.

6.4 Docker e Docker Compose

Os Dockerfiles de frontend e backend deverão utilizar multi-stage build. O frontend será compilado em imagem Node e servido por Nginx; o backend será compilado com Maven/Gradle e executado em imagem JRE Java 21. O Compose será utilizado para desenvolvimento local com frontend, backend e PostgreSQL.

```
# Backend — exemplo multi-stage
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn -q -DskipTests package

FROM eclipse-temurin:21-jre
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java","-jar","app.jar"]

# Frontend — exemplo multi-stage
FROM node:22-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
```

6.5 Pipeline CI/CD

1. Lint e verificação estática do frontend e backend.

2. Execução dos testes unitários e de integração.
3. Build das aplicações e imagens Docker.
4. Análise de dependências e imagem (SCA/container scan).
5. Publicação automática em staging.
6. Smoke tests de login, reserva e consulta financeira.
7. Aprovação manual e promoção para produção.
8. Verificação pós-deploy; rollback se health checks ou smoke tests falharem.

Critério de bloqueio: pipeline não promove para staging/produção se build falhar, testes críticos falharem, vulnerabilidade crítica/alta sem exceção aprovada for encontrada ou smoke test não passar.

6.6 Segurança — controles OWASP

Risco OWASP / domínio	Controle adotado
Broken Access Control	RBAC no backend, validação de propriedade de unidade e testes negativos.
Cryptographic Failures	TLS 1.2+, hash adaptativo de senha, criptografia de dados sensíveis e backups.
Injection	Consultas parametrizadas/JPA, validação e escaping de saída.
Insecure Design	Threat modeling, revisão de fluxos críticos e limites de negócio.
Security Misconfiguration	Hardening, portas mínimas, headers seguros e remoção de credenciais padrão.
Vulnerable Components	SCA no pipeline, atualização e inventário de dependências.
Authentication Failures	JWT com expiração, proteção contra brute force e política de senha.
Software/Data Integrity Failures	Pull Requests, revisão, assinatura e pipeline controlado.
Logging/Monitoring Failures	Logs estruturados, trilhas de auditoria e alertas.
SSRF	Allowlist de destinos externos, bloqueio de rede interna e validação de URLs.

6.7 Observabilidade

- Logs estruturados JSON com correlation ID, sem senhas, tokens ou documentos em texto puro.
- Métricas técnicas: latência p50/p95/p99, taxa de erro, throughput, CPU, memória, conexões e saúde do banco.
- Métricas de negócio: reservas criadas/canceladas, autorizações de visitantes, ocorrências abertas e cobranças consultadas.
- Alertas: disponibilidade, erro 5xx, latência acima do limite, falha de backup e esgotamento de recursos.
- Ferramentas previstas: Spring Boot Actuator, Prometheus e Grafana.

BLOCO 7 — PLANEJAMENTO E ESTIMATIVAS

7.1 WBS / EAP — três níveis

Nível 1	Nível 2	Nível 3 — pacotes de trabalho
1. Planejamento	1.1 Iniciação	1.1.1 Visão; 1.1.2 escopo; 1.1.3 stakeholders.
2. Análise e projeto	2.1 Requisitos / 2.2 Arquitetura / 2.3 Dados	2.1.1 RF/RNF; 2.1.2 histórias; 2.2.1 ADR/C4; 2.2.2 UML; 2.3.1 ER; 2.3.2 DDL.
3. Construção	3.1 Frontend / 3.2 Backend / 3.3 Integração	3.1.1 telas; 3.1.2 responsividade; 3.2.1 autenticação; 3.2.2 módulos; 3.3.1 API/DB.
4. Infraestrutura	4.1 Ambientes / 4.2 CI/CD / 4.3 Segurança	4.1.1 rede; 4.1.2 containers; 4.2.1 pipeline; 4.3.1 secrets/monitoramento.
5. Testes	5.1 Unitários / 5.2 Integração / 5.3 E2E	5.1.1 regras; 5.2.1 persistência; 5.3.1 fluxos principais.
6. Entrega	6.1 Homologação / 6.2 Publicação / 6.3 Documentação	6.1.1 aceite; 6.2.1 deploy; 6.3.1 README e treinamento.

7.2 Estimativa de esforço e cronograma

O backlog inicial totaliza 74 story points. Para planejamento, considera-se velocidade inicial de 20 pontos por sprint de duas semanas, resultando em quatro sprints (oito semanas), com 74 pontos de trabalho e 6 pontos de capacidade não comprometida. A velocidade é hipótese e será recalibrada após a Sprint 1.

Sprint	Pontos planejados	Entregas principais
1	19	Autenticação, usuários, unidades, funcionários e base de navegação.
2	20	Visitantes, acessos, reservas e comunicados.
3	20	Ocorrências, solicitações de manutenção e financeiro.
4	15	Notificações, relatórios, testes integrados, correções e preparação de release.
Total	74	Quatro sprints de duas semanas.

7.3 Equipe e pesquisa salarial

Para estimativa de custo profissional, considera-se equipe de cinco pessoas. Os valores de referência foram consultados em páginas de salários do Glassdoor Brasil em 08/11/2026; são médias/faixas indicativas, não proposta de contratação nem salário garantido.

Perfil	Qtd.	Referência mensal bruta	Fonte / critério
Gerente de projetos de TI / PO	1	R$ 8.000	Glassdoor: média base aproximada exibida para gerente de projetos em TI.
Desenvolvedor backend Java	1	R$ 8.000	Glassdoor: média base aproximada para Java Backend.
Desenvolvedor frontend React	1	R$ 4.000	Glassdoor: média base aproximada para Frontend React.

Analista QA	1	R$ 5.000	Glassdoor: média base aproximada para Quality Assurance Analyst.
Engenheiro DevOps	1	R$ 7.000	Glassdoor: média base aproximada para Engenheiro DevOps.
Total	5	R$ 32.000/mês	Soma das referências mensais.

7.4 Custo consolidado do projeto

Item	Premissa	Estimativa
Equipe — remuneração bruta	R$ 32.000 x 2 meses	R$ 64.000
Equipe — custo provisionado	R$ 64.000 x 1,7	R$ 108.800
Infraestrutura cloud	US$ 241 x 2 meses	US$ 482
Contingência	10% aplicado separadamente	R$ 10.880 + US$ 48,20
Total estimado	Sem conversão cambial	R$ 119.680 + US$ 530,20

7.5 Matriz de riscos — projeto

Risco	Categoria	Prob.	Impacto	Mitigação
Vazamento de dados pessoais	Segurança/regulatório	Média	Alto	Criptografia, RBAC, auditoria e avaliação LGPD.
Falha de failover do banco	Técnico	Baixa	Alto	Testes de recuperação e simulação de indisponibilidade.
Conflito de reserva simultânea	Técnico	Média	Alto	Transação, constraint de intervalo e testes concorrentes.
Atraso na integração frontend/API	Equipe/técnico	Média	Médio	Contrato OpenAPI, mocks e integração contínua.
Estimativa salarial acima do orçamento	Financeiro	Média	Médio	Revisão de senioridade, escopo e duração.
Baixa adesão dos moradores	Negócio	Média	Médio	Treinamento, comunicação e piloto com usuários.
Mudança de escopo durante sprint	Gestão	Média	Médio	Backlog priorizado e controle de mudanças.
Dependência ou vulnerabilidade crítica	Segurança	Média	Alto	SCA, atualização e bloqueio de pipeline.
Perda de integrante-chave	Equipe	Baixa	Médio	Documentação, revisão cruzada e compartilhamento de conhecimento.

Mudança regulatória ou retenção inadequada	Regulatório	Baixa	Alto	Revisão jurídica e política de retenção documentada.

BLOCO 8 — DOCUMENTAÇÃO PARA DESENVOLVIMENTO

8.1 README — conteúdo mínimo

Nome: CondoSmart — Plataforma Inteligente de Gestão de Condomínios. Descrição: aplicação web para gestão de moradores, unidades, visitantes, reservas, comunicados, ocorrências, funcionários e cobranças.

Stack: React + TypeScript; Java 21 + Spring Boot; PostgreSQL; Docker; GitHub Actions; Flyway; OpenAPI 3.0.

Estrutura do repositório

```
condosmart/
  frontend/
    src/
    public/
    Dockerfile
  backend/
    src/
    pom.xml
    Dockerfile
  database/
    migrations/
  docs/
    arquitetura/
    uml/
    api/
  docker-compose.yml
  .env.example
  README.md
```

8.2 Variáveis de ambiente

```
APP_PORT=8080
DB_URL=jdbc:postgresql://database:5432/condosmart
DB_USER=condosmart_user
DB_PASSWORD=change_me
JWT_SECRET=replace_with_secret_from_secret_manager
JWT_EXPIRATION=3600000
CORS_ALLOWED_ORIGINS=https://app.condosmart.example
```

O arquivo .env.example contém nomes e exemplos não secretos. O arquivo .env real não deverá ser versionado. Em produção, as credenciais deverão vir de serviço de gerenciamento de segredos.

8.3 Contrato da API — OpenAPI 3.0

Método	Endpoint	Descrição	Autorização
POST	/api/auth/login	Autenticar usuário.	Público
GET / POST	/api/usuarios	Consultar / cadastrar usuários.	ADMIN
GET / POST	/api/unidades	Consultar / cadastrar unidades.	ADMIN/SINDICO
POST	/api/visitantes/autorizacoes	Autorizar visitante.	MORADOR

POST	/api/acessos/entrada	Registrar entrada.	PORTEIRO
POST	/api/acessos/saida	Registrar saída.	PORTEIRO
GET	/api/areas-comuns	Listar áreas disponíveis.	Usuário autenticado
GET / POST	/api/reservas	Consultar / criar reserva.	MORADOR
PATCH	/api/reservas/{id}/cancelamento	Cancelar reserva.	Proprietário/SINDICO
POST	/api/ocorrencias	Registrar ocorrência.	MORADOR
GET / PATCH	/api/ocorrencias	Consultar / atualizar status.	MORADOR/SINDICO
GET / POST	/api/cobrancas	Consultar / cadastrar cobranças.	MORADOR/SINDICO
POST / GET	/api/comunicados	Publicar / listar comunicados.	SINDICO / autenticado
GET	/api/notificacoes	Consultar notificações.	Usuário autenticado

Exemplo OpenAPI 3.0

```
openapi: 3.0.3
info:
  title: CondoSmart API
  version: 1.0.0
servers:
  - url: https://api.condosmart.example
paths:
  /api/auth/login:
    post:
      summary: Autenticar usuário
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              required: [email, senha]
              properties:
                email: { type: string, format: email }
                senha: { type: string, format: password }
      responses:
        '200': { description: Token emitido }
        '401': { description: Credenciais inválidas }
  /api/reservas:
    post:
      security:
        - bearerAuth: []
      summary: Criar reserva
      responses:
        '201': { description: Reserva criada }
        '409': { description: Conflito de horário }
        '401': { description: Não autenticado }
components:
  securitySchemes:
    bearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT
```

O contrato completo deverá incluir schemas de requisição/resposta, parâmetros, exemplos, códigos de erro, paginação e regras de autorização para todos os endpoints.

8.4 Guia de padrões de código

- Java: classes em PascalCase; métodos/variáveis em camelCase; camadas Controller, Service, Repository, DTO e Entity separadas.

- Não colocar regra de negócio nos controllers; validar entradas com Bean Validation.
- React: componentes reutilizáveis, TypeScript, páginas e serviços separados; centralizar chamadas HTTP.
- Tratar erros com formato padronizado e não retornar stack trace ao cliente.
- Criar testes para regras de negócio e revisar código por Pull Request.

8.5 Git workflow

Branches: main (produção), develop (integração), feature/nome-da-funcionalidade e fix/nome-do-problema. O fluxo exige branch de trabalho, commits descritivos, Pull Request, revisão, pipeline verde e aprovação antes do merge. Releases devem ser identificadas por tags versionadas.

8.6 Sprint 1 detalhada

Objetivo: entregar autenticação e cadastros básicos. Histórias previstas: HU001 (3 pontos), HU002 (3), HU003 (5), HU011 (5) e HU007 (3), totalizando 19 pontos. HU007 depende da existência do módulo de reservas; portanto, se não houver reservas previamente cadastradas, poderá ser substituída por tarefa técnica de fundação ou movida para sprint seguinte. Essa dependência deve ser resolvida no Sprint Planning.

História	Critérios de aceite resumidos	Pontos
HU001	Cadastro válido; e-mail duplicado recusado; dados obrigatórios validados.	3
HU002	Unidade cadastrada; identificação duplicada no mesmo condomínio recusada.	3
HU003	Login válido emite token; inválido retorna 401; perfil é carregado.	5
HU011	Síndico cadastra/atualiza funcionário; operação fica auditada.	5
HU007	Morador cancela apenas reserva própria e dentro do prazo; status atualizado.	3

BLOCO 9 — QUALIDADE E PROCESSO

9.1 Modelo de domínio e glossário

Termo	Definição
Condomínio residencial	Conjunto de unidades residenciais administradas pela plataforma.
Unidade	Apartamento ou casa identificada no condomínio.
Morador	Pessoa vinculada a uma ou mais unidades.
Síndico	Responsável pela administração do condomínio.
Porteiro	Responsável por consultar autorizações e registrar acessos.
Funcionário	Profissional que executa atividades do condomínio.

Visitante	Pessoa externa autorizada a entrar.
Autorização	Registro de permissão de acesso com período de validade.
Registro de acesso	Evento de entrada ou saída de visitante.
Área comum	Espaço compartilhado que pode ser reservado.
Reserva	Agendamento de uma área comum por intervalo de tempo.
Ocorrência	Problema comunicado por morador.
Cobrança	Valor associado a uma unidade e vencimento.
Notificação	Mensagem de atualização direcionada a usuário.

9.2 Estratégia de testes

Nível	Objetivo	Exemplos	Ferramentas
Unitário	Testar regras isoladas.	Validação de reserva, permissões, cálculo de cobrança e status.	JUnit, Mockito, Vitest.
Integração	Testar integração entre componentes e banco.	Persistência, login, transação de reserva, consulta financeira.	Spring Boot Test, Testcontainers.
E2E	Validar fluxos completos pela interface.	Login, autorização de visitante, reserva, cobrança e ocorrência.	Playwright.

9.3 Metas de cobertura e qualidade

Indicador	Meta inicial	Como medir
Cobertura de linhas no backend	≥ 80%	JaCoCo no pipeline.
Cobertura de regras críticas	≥ 90%	Relatório de cobertura focado em regras de autorização, reservas e cobrança.
Fluxos críticos E2E	100% dos fluxos principais definidos	Suíte Playwright em staging.
Vulnerabilidades críticas/altas	0 abertas sem exceção aprovada	SCA e análise de imagens.
Desempenho API	p95 ≤ 300 ms em carga de referência	Teste de carga e métricas.

9.4 Definition of Ready (DoR)

- História descrita com persona, ação e objetivo.
- Critérios de aceitação Given/When/Then definidos.
- Prioridade MoSCoW e estimativa registradas.
- Dependências e regras de negócio identificadas.
- Design, contrato ou dados necessários disponíveis.
- História suficientemente pequena para caber na sprint.

9.5 Definition of Done (DoD)

- Implementação concluída e revisada por outro integrante.
- Critérios de aceitação atendidos.

- Testes unitários e de integração relevantes aprovados.
- Pipeline executado sem falhas bloqueadoras.
- Sem vulnerabilidades críticas/altas sem exceção aprovada.
- Documentação e contrato da API atualizados.
- Funcionalidade implantada em staging e validada.

9.6 Metodologia — Scrum

Scrum foi escolhido por permitir planejamento incremental em sprints de duas semanas, priorização contínua do backlog e inspeção frequente dos resultados. É adequado porque os módulos podem ser entregues progressivamente e os requisitos podem ser refinados com feedback dos stakeholders.

Evento / papel	Aplicação no CondoSmart
Sprint Planning	Selecionar histórias e definir objetivo da sprint.
Daily Scrum	Inspecionar progresso e impedimentos diariamente.
Sprint Review	Demonstrar incrementos aos stakeholders.
Sprint Retrospective	Definir melhorias para o próximo ciclo.
Product Owner	Priorizar valor e manter backlog ordenado.
Scrum Master	Facilitar o processo e remover impedimentos.
Developers	Projetar, desenvolver, testar e entregar incremento.

9.7 Melhoria contínua

A equipe analisará métricas de qualidade, falhas, tempo de ciclo, retrabalho e feedback dos usuários ao fim de cada sprint. Ações de melhoria serão priorizadas no backlog. Incidentes em produção deverão gerar análise de causa e plano de prevenção.